

POLITECHNIKA WROCLAWSKA  Wydział Informatyki i Telekomunikacji	Wydział: Informatyki i Telekomunikacji Kierunek: Informatyka stosowana st. II Rok Akademicki: 2025/2026 Semestr: Letni Termin: <i>Pn 13:15 – 15:00</i>
Projektowanie i wdrażanie systemów w chmurze Raport z projektu 1	
Data oddania raportu: 06.04.2025	Prowadzący: dr inż. Rafał Palak Autor: Krzysztof Perdek

Spis treści

1.	Konteneryzacja aplikacji	2
2.	Funkcja lambda do automatycznego zatwierdzenia rejestracji	2
3.	Konfiguracja Terraform	3
4.	Infrastruktura AWS	9
5.	Prezentacja aplikacji.....	11
6.	Ręczne skonfigurowanie infrastruktury	12

1. Konteneryzacja aplikacji

Aplikacja jest podzielona na klient i serwer, które zostały wdrożone przy poprzez utworzenie obrazu Docker. Obrazy aplikacji zostały zbudowane w sposób ręczny i udostępnione poprzez AWS Elastic Container Registry.

```
C:\Users\Krzys\Desktop\Programowanie\projekt-chmury\text-messenger-client>docker build -t client .  
[+] Building 0.0s (0/0) docker:default  
[+] Building 3.2s (31/31) FINISHED
```

Rysunek 1 Budowa obrazu aplikacji klienta

```
C:\Users\Krzys\Desktop\Programowanie\projekt-chmury\text-messenger-client>aws ecr get-login-password --region us-east-1  
| docker login --username AWS --password-stdin 733338247552.dkr.ecr.us-east-1.amazonaws.com  
Login Succeeded  
  
C:\Users\Krzys\Desktop\Programowanie\projekt-chmury\text-messenger-client>docker tag client 733338247552.dkr.ecr.us-east-1.amazonaws.com/264127/app:latest  
  
C:\Users\Krzys\Desktop\Programowanie\projekt-chmury\text-messenger-client>docker push 733338247552.dkr.ecr.us-east-1.amazonaws.com/264127/app:latest  
The push refers to repository [733338247552.dkr.ecr.us-east-1.amazonaws.com/264127/app]  
ce1a97ea1352: Layer already exists  
580620b90f52: Layer already exists  
07080d50b3fd: Layer already exists  
46379307e720: Layer already exists  
47fb8e6ca0fc: Layer already exists  
4c8c416990ce: Layer already exists  
6d190562115f: Layer already exists  
latest: digest: sha256:539c5f61c2eab35401f924bda5e3885c67dca07b7d6c11a043124a0ab0961335 size: 1782
```

Rysunek 2 Wgranie obrazu klienta do AWS ECR

Analogiczne kroki zostały wykonane dla serwera.

Private repositories (2)

Search by repository substring

Repository name	URI	Created at	Tag immutability	Encryption type
264127/app	733338247552.dkr.ecr.us-east-1.amazonaws.com/264127/app	March 22, 2025, 15:04:16 (UTC+01)	Mutable	AES-256
264127/server	733338247552.dkr.ecr.us-east-1.amazonaws.com/264127/server	March 22, 2025, 19:31:49 (UTC+01)	Mutable	AES-256

Rysunek 3 Widok AWS ECR

2. Funkcja lambda do automatycznego zatwierdzenia rejestracji

Aplikacja posiadała wcześniej własny system autoryzacji, który nie zakładał mailowego potwierdzenia rejestracji konta. W celu zachowania wcześniejszego zachowania aplikacji, została utworzona funkcja lambda, która zostanie zastosowana jako Pre-Signup hook w AWS Cognito.

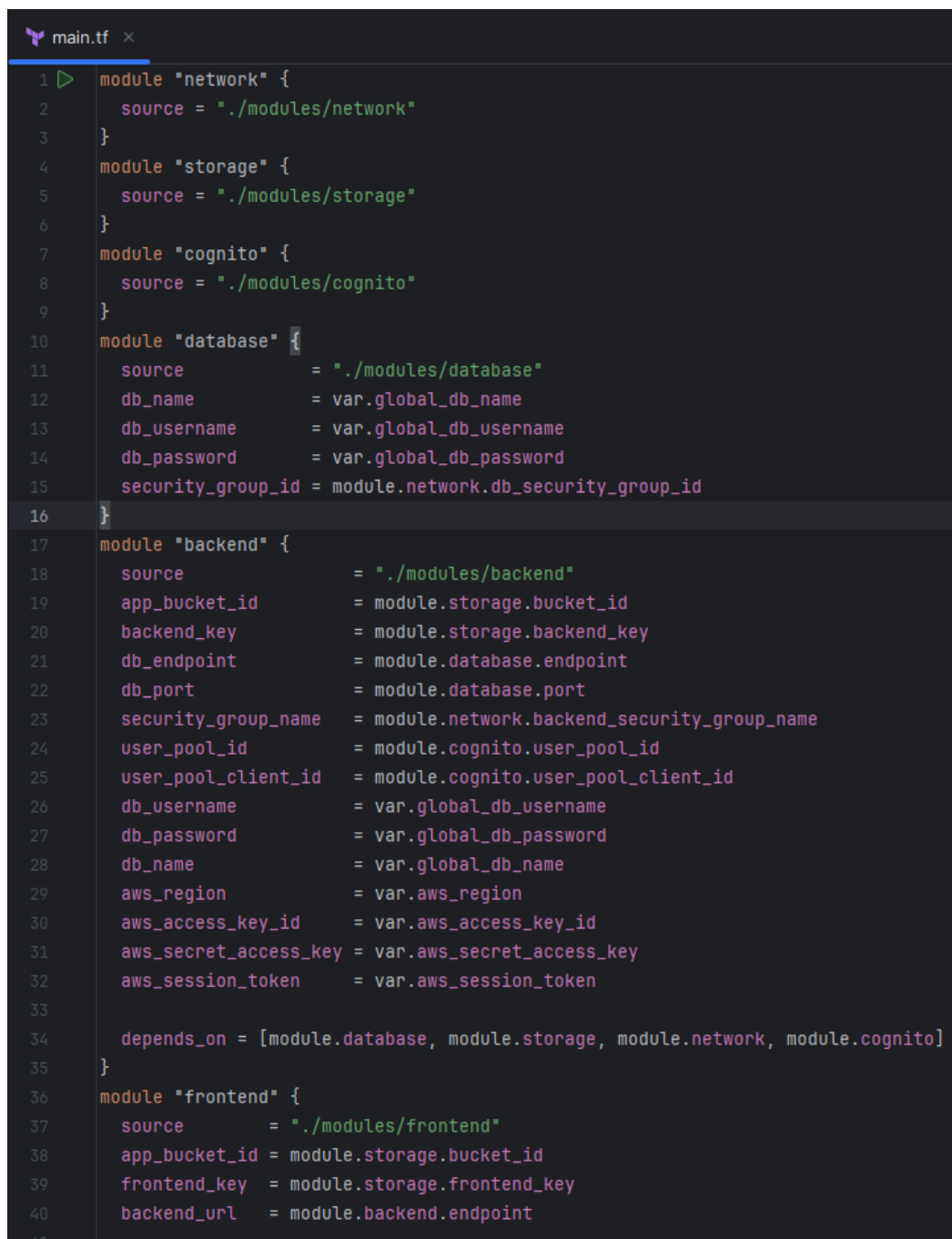


Rysunek 4 Kod źródłowy funkcji lambda

3. Konfiguracja Terraform

Konfiguracja terraform została podzielona na pięć modułów:

- Network – konfiguracja Security Groups użytych w konfiguracji bazy danych i backend'u.
- Storage – konfiguracja S3 bucket
- Cognito – konfiguracja Cognito
- Database – konfiguracja bazy danych
- Backend – konfiguracja backend'u
- Frontend – konfiguracja frontend'u.



```
main.tf x
1 module "network" {
2   source = "../modules/network"
3 }
4 module "storage" {
5   source = "../modules/storage"
6 }
7 module "cognito" {
8   source = "../modules/cognito"
9 }
10 module "database" {
11   source = "../modules/database"
12   db_name = var.global_db_name
13   db_username = var.global_db_username
14   db_password = var.global_db_password
15   security_group_id = module.network.db_security_group_id
16 }
17 module "backend" {
18   source = "../modules/backend"
19   app_bucket_id = module.storage.bucket_id
20   backend_key = module.storage.backend_key
21   db_endpoint = module.database.endpoint
22   db_port = module.database.port
23   security_group_name = module.network.backend_security_group_name
24   user_pool_id = module.cognito.user_pool_id
25   user_pool_client_id = module.cognito.user_pool_client_id
26   db_username = var.global_db_username
27   db_password = var.global_db_password
28   db_name = var.global_db_name
29   aws_region = var.aws_region
30   aws_access_key_id = var.aws_access_key_id
31   aws_secret_access_key = var.aws_secret_access_key
32   aws_session_token = var.aws_session_token
33
34   depends_on = [module.database, module.storage, module.network, module.cognito]
35 }
36 module "frontend" {
37   source = "../modules/frontend"
38   app_bucket_id = module.storage.bucket_id
39   frontend_key = module.storage.frontend_key
40   backend_url = module.backend.endpoint
41 }
```

Rysunek 5 Konfiguracja wejściowa main.tf

```
modules\cognito\main.tf x
1 resource "aws_cognito_user_pool" "pool" {
2   name = "app-user-pool"
3
4   lambda_config {
5     pre_sign_up = var.lambda_script_arn
6   }
7
8   password_policy {
9     minimum_length      = 8
10    require_uppercase   = true
11    require_lowercase   = true
12    require_numbers     = true
13    require_symbols     = false
14  }
15 }
16
17 resource "aws_lambda_permission" "allow_cognito" {
18   statement_id = "AllowCognitoToInvokeFunction"
19   action       = "lambda:InvokeFunction"
20   function_name = var.lambda_script_arn
21   principal    = "cognito-idp.amazonaws.com"
22   source_arn   = aws_cognito_user_pool.pool.arn
23 }
24
25 resource "aws_cognito_user_pool_client" "client" {
26   name = "user-pool-client"
27
28   user_pool_id = aws_cognito_user_pool.pool.id
29
30   explicit_auth_flows = [
31     "ALLOW_USER_PASSWORD_AUTH",
32     "ALLOW_REFRESH_TOKEN_AUTH"
33   ]
34 }
35
```

Utworzenie user poola

Przypisanie lambda hooka, który automatycznie, zatwierdzi rejestracji.

Polityka haseł

Zezwolenie Cognito na wykorzystanie funkcji lambda

Utworzenie klienta user pool

Rysunek 6 Konfiguracja modules/cognito/main.tf

```
main.tf  modules\backend\main.tf  modules\network\main.tf ×
1 resource "aws_security_group" "backend_sg" { Security group backend'u
2   name      = "backend-security-group"
3   description = "Security group for backend server"
4
5   ingress {
6     from_port = 80 Dostęp HTTP z zewnątrz
7     to_port   = 80
8     protocol  = "tcp"
9     cidr_blocks = ["0.0.0.0/0"]
10  }
11  ingress {
12    from_port = 443 Dostęp HTTPS z zewnątrz
13    to_port   = 443
14    protocol  = "tcp"
15    cidr_blocks = ["0.0.0.0/0"]
16  }
17  egress {
18    from_port = 0
19    to_port   = 0
20    protocol  = "-1"
21    cidr_blocks = ["0.0.0.0/0"]
22  }
23 }
24
25 resource "aws_security_group" "db_sg" { Security group bazy danych
26   name      = "db-security-group"
27   description = "Security group for RDS database"
28
29   ingress {
30     from_port = 5432 Dostęp do PostgreSQL dla urządzeń w security group backend'u
31     to_port   = 5432
32     protocol  = "tcp"
33     security_groups = [aws_security_group.backend_sg.id]
34   }
35   egress {
36     from_port = 0
37     to_port   = 0
38     protocol  = "-1"
39     cidr_blocks = ["0.0.0.0/0"]
40   }
41 }
```

Rysunek 7 Konfiguracja modules/network/main.tf

```

main.tf modules\storage\main.tf × modules\backend\main.tf modules\network\main.tf
1 resource "aws_s3_bucket" "app_bucket" {
2     bucket = var.bucket_name
3 }
4
5 resource "aws_s3_object" "frontend_dockerrun" {
6     bucket = aws_s3_bucket.app_bucket.id
7     key    = "beanstalk/frontend/Dockerrun.aws.json"
8     content = jsonencode({
9         "AWSEBDockerrunVersion" = "1"
10        "Image" = {
11            "Name" = "733338247552.dkr.ecr.us-east-1.amazonaws.com/264127/app:latest"
12            "Update" = "true"
13        }
14        "Ports" = [
15            { "ContainerPort" = "4000" }
16        ]
17    })
18 }
19
20 resource "aws_s3_object" "backend_dockerrun" {
21     bucket = aws_s3_bucket.app_bucket.id
22     key    = "beanstalk/backend/Dockerrun.aws.json"
23     content = jsonencode({
24         "AWSEBDockerrunVersion" = "1"
25        "Image" = {
26            "Name" = "733338247552.dkr.ecr.us-east-1.amazonaws.com/264127/server:latest"
27            "Update" = "true"
28        }
29        "Ports" = [
30            { "ContainerPort" = "4000" }
31        ]
32    })
33 }

```

Plik Dockerrun uruchamiający frontend z obrazu w ECR.

Wskazanie portu, który nasłuchuje skonteneryzowana aplikacja

Plik Dockerrun uruchamiający backend z obrazu w ECR

Rysunek 8 Konfiguracja modules/storage/main.tf

```

main.tf modules\storage\main.tf modules\database\main.tf ×
1 resource "aws_db_instance" "postgres" {
2     identifier      = "text-messenger-db"
3     engine          = "postgres"
4     engine_version  = "17.4"
5     instance_class  = "db.t3.micro"
6     allocated_storage = 20
7     username        = var.db_username
8     password        = var.db_password
9     db_name          = var.db_name
10    vpc_security_group_ids = [var.security_group_id]
11    skip_final_snapshot = true
12    #publicly_accessible = true
13 }

```

Baza PostgreSQL

Rysunek 9 Konfiguracja modules/database/main.tf

```
main.tf  modules\...\main.tf x
1 resource "aws_elastic_beanstalk_application" "backend" {
2   name      = "backend-app"
3   description = "Chat application backend"
4 }
5 resource "aws_elastic_beanstalk_application_version" "backend_version" {
6   name      = "backend-v1"
7   application = aws_elastic_beanstalk_application.backend.name
8   bucket     = var.app_bucket_id      Wersja aplikacji jest tworzona z pliku Dockerrun.aws.json
9   key        = var.backend_key        z S3 Bucket
10 }
11 resource "aws_elastic_beanstalk_environment" "backend_env" {
12   name      = "backend-app-env"
13   application = aws_elastic_beanstalk_application.backend.name
14   solution_stack_name = "64bit Amazon Linux 2023 v4.4.4 running Docker"
15   version_label = aws_elastic_beanstalk_application_version.backend_version.name
16   setting {
17     namespace = "aws:elasticbeanstalk:environment"
18     name      = "EnvironmentType"      Utworzenie pojedynczej instancji
19     value     = "SingleInstance"
20   }
21   setting {
22     namespace = "aws:autoscaling:launchconfiguration"
23     name      = "SecurityGroups"      Przypisanie security group, który pozwoli na dostęp
24     value     = var.security_group_name do bazy
25   }
26   setting {
27     namespace = "aws:autoscaling:launchconfiguration"
28     name      = "IamInstanceProfile"  Profil instancji, wymagany do
29     value     = "LabInstanceProfile" uruchomienia środowiska
30   }
31   setting {
32     namespace = "aws:elasticbeanstalk:application:environment"      Zmienna środowiska wskazująca
33     name      = "DATABASE_URL"      aplikację adres do bazy danych
34     value     = "ecto://${var.db_username}:${var.db_password}@${var.db_endpoint}:${var.db_port}/${var.db_name}"
35   }
36   setting {
37     namespace = "aws:elasticbeanstalk:application:environment"
38     name      = "AWS_USER_POOL_ID"  Przekazanie aplikacji ID user poolu
39     value     = var.user_pool_id
40   }
41   setting {
```

Rysunek 10 Konfiguracja modules/backend/main.tf 1/2

```

11 resource "aws_elasticbeanstalk_environment" "backend_env" {
26   setting {
30   }
31   setting {
32     namespace = "aws:elasticbeanstalk:application:environment"
33     name      = "DATABASE_URL"
34     value     = "ecto://${var.db_username}:${var.db_password}@${var.db_endpoint}:${var.db_port}/${var.db_name}"
35   }
36   setting {
37     namespace = "aws:elasticbeanstalk:application:environment"
38     name      = "AWS_USER_POOL_ID"
39     value     = var.user_pool_id
40   }
41   setting {
42     namespace = "aws:elasticbeanstalk:application:environment"
43     name      = "AWS_COGNITO_CLIENT_ID"
44     value     = var.user_pool_client_id
45   }
46   setting {
47     namespace = "aws:elasticbeanstalk:application:environment"
48     name      = "AWS_ACCESS_KEY_ID"
49     value     = var.aws_access_key_id
50   }
51   setting {
52     namespace = "aws:elasticbeanstalk:application:environment"
53     name      = "AWS_SECRET_ACCESS_KEY"
54     value     = var.aws_secret_access_key
55   }
56   setting {
57     namespace = "aws:elasticbeanstalk:application:environment"
58     name      = "AWS_SESSION_TOKEN"
59     value     = var.aws_session_token
60   }
61   setting {
62     namespace = "aws:elasticbeanstalk:application:environment"
63     name      = "AWS_REGION"
64     value     = var.aws_region
65   }
66 }

```

Przekazanie aplikacji ID klienta user poola

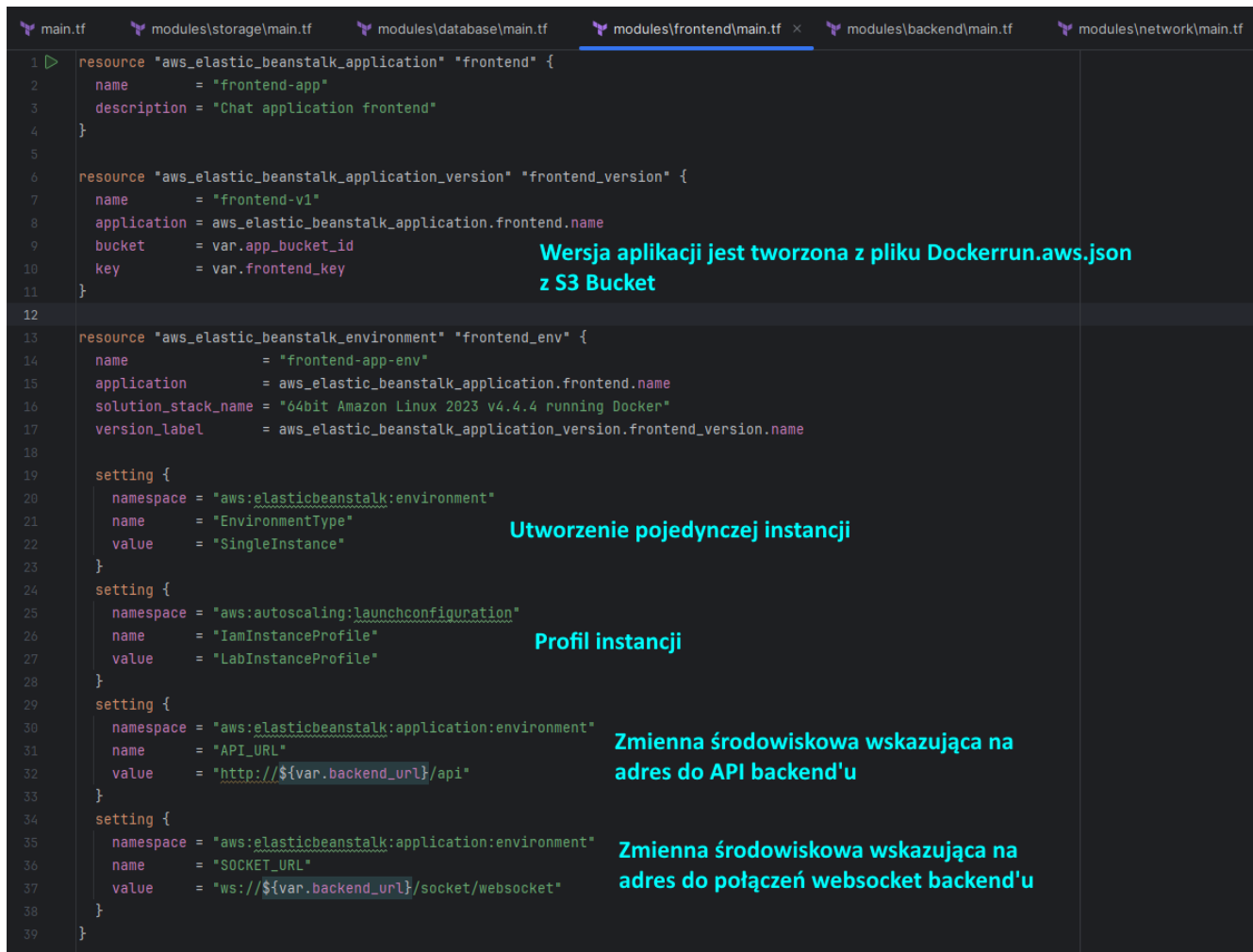
Przekazanie aplikacji access key ID (do zapytań API)

Przekazanie aplikacji access key (do zapytań API)

Przekazanie aplikacji tokenu sesji (do zapytań API)

Przekazanie aplikacji regionu AWS w którym działa

Rysunek 11 Konfiguracja modules/backend/main.tf 2/2



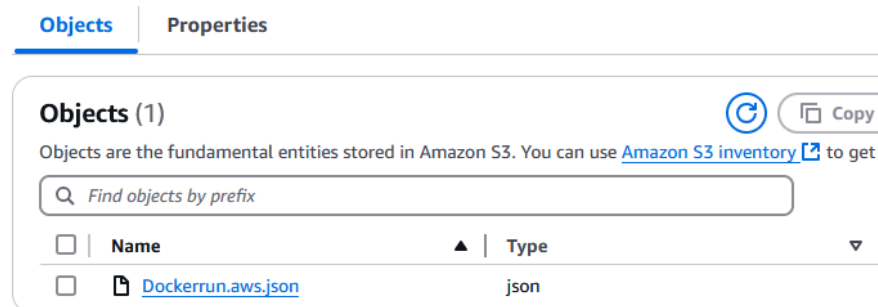
Rysunek 12 Konfiguracja modules/frontend/main.tf

4. Infrastruktura AWS

Po zastosowaniu konfiguracji Terraform utworzona zostaje następująca infrastruktura:

≡ [Amazon S3](#) > [Buckets](#) > [264127.application](#) > [beanstalk/](#) > [backend/](#)

backend/



Rysunek 13 S3 z konfiguracją Dockerrun dla backendu

frontend/

Objects

Properties

Objects (1)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get

Find objects by prefix

☐	Name	Type
☐	Dockerrun.aws.json	json

Rysunek 14 S3 z konfiguracją Dockerrun dla frontendu

Security Groups (4)

☐	Name	Security group ID	Security group name	Description
☐	-	sg-0601134da346ca5b7	db-security-group	Security group for RDS database
☐	-	sg-0c33c357504eebf28	default	default VPC security group
☐	backend-app-env	sg-0d964caad7e15b60b	awseb-e-imuw3hxmdu-stack-AWSEBSecurityGroup-7MaPMaQsRlrc	SecurityGroup for ElasticBeanstalk envir...
☐	-	sg-0cc1721d1858ecae1	backend-security-group	Security group for backend server

Rysunek 15 Security groups: dwa z konfiguracji + backend-app-env utworzony automatycznie wraz ze środowiskiem

Databases (1)

☐	DB identifier	Status	Role	Engine	Region ...	Size
☐	text-messenger-db	Available	Instance	PostgreSQL	us-east-1a	db.t3.micro

Rysunek 16 Instancja z bazą danych PostgreSQL

Environments (2)

☐	Environment name	Health	Application ...	Platform	Domain	Running ver...
☐	backend-app-env	Ok	backend-app	Docker running...	backend-app-env.eba-yakpze7...	backend-v1
☐	frontend-app-env	Ok	frontend-app	Docker running...	frontend-app-env.eba-rh4akpz...	frontend-v1

Rysunek 17 Środowiska dla aplikacji frontend oraz aplikacji backend

Applications (2)

☐	Application name	Environments
☐	backend-app	backend-app-env
☐	frontend-app	frontend-app-env

Rysunek 18 Aplikacje frontend i backend

Instances (2) [Info](#)

All states ▾

Instance state = running X Clear filters

☐	Name ✎	Instance ID	Instance state	Instance type	Status check
☐	frontend-app-env	i-0742d5cc4357ca150	Running 🔍 🔍	t3.micro	3/3 checks passed
☐	backend-app-env	i-0e3329bbea87af8e7	Running 🔍 🔍	t3.micro	3/3 checks passed

Rysunek 19 Instancje EC2 utworzone przez środowiska

User pools (1) [Info](#)

View and configure your user pools. User pools are directories of federated and local user profiles. They provide authentication options for your users.

< 1 > ⚙️

☐	User pool name	User pool ID	Created time	Last updated time
☐	app-user-pool	us-east-1_VB9nZZv7d	2 hours ago	3 minutes ago

Rysunek 20 Pula użytkowników

App clients and analytics [Info](#)

App clients (1) [Info](#)

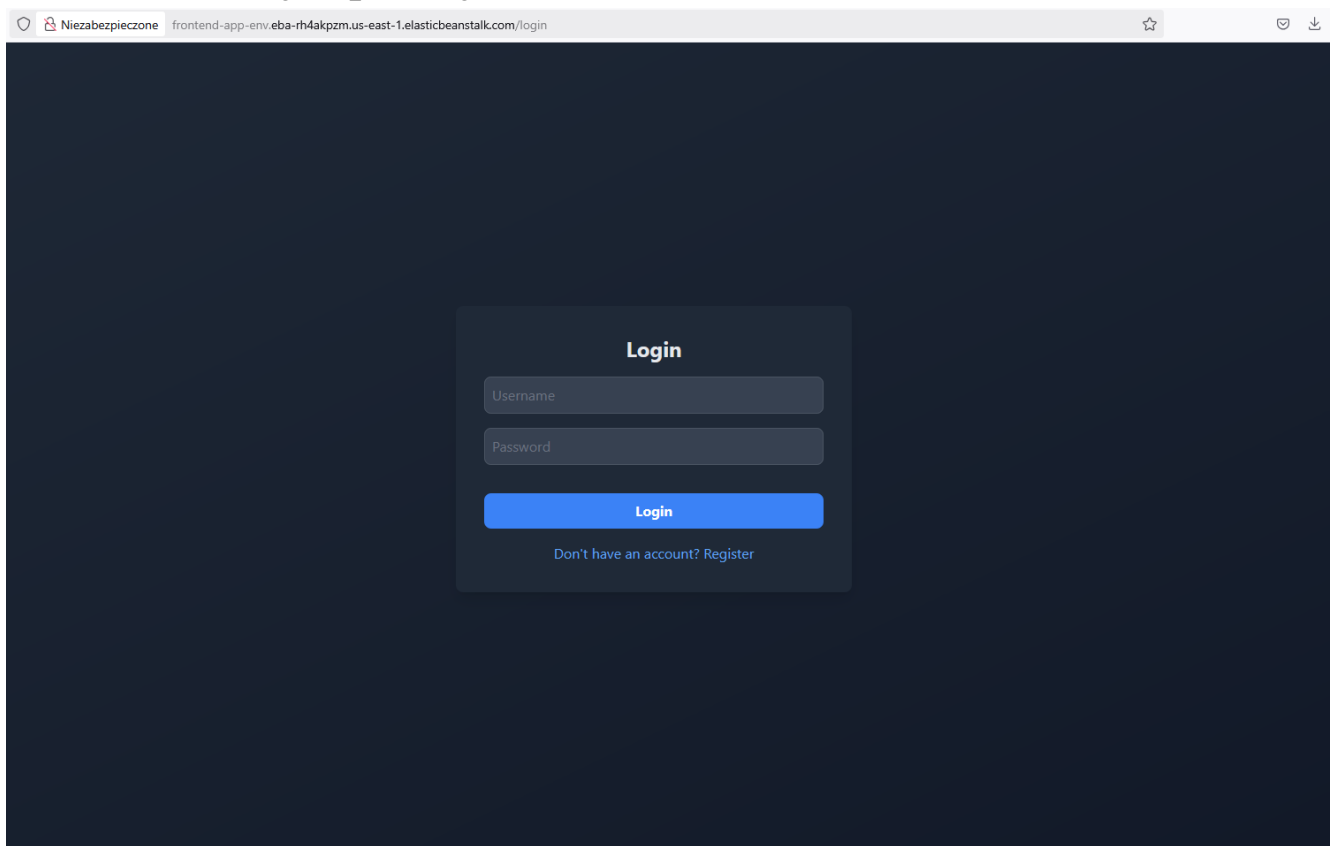
Configure an app client. App clients are the user pool authentication resources attached to your app. Select an app client to configure the permitted authentication actions for an app.

< 1 > ⚙️

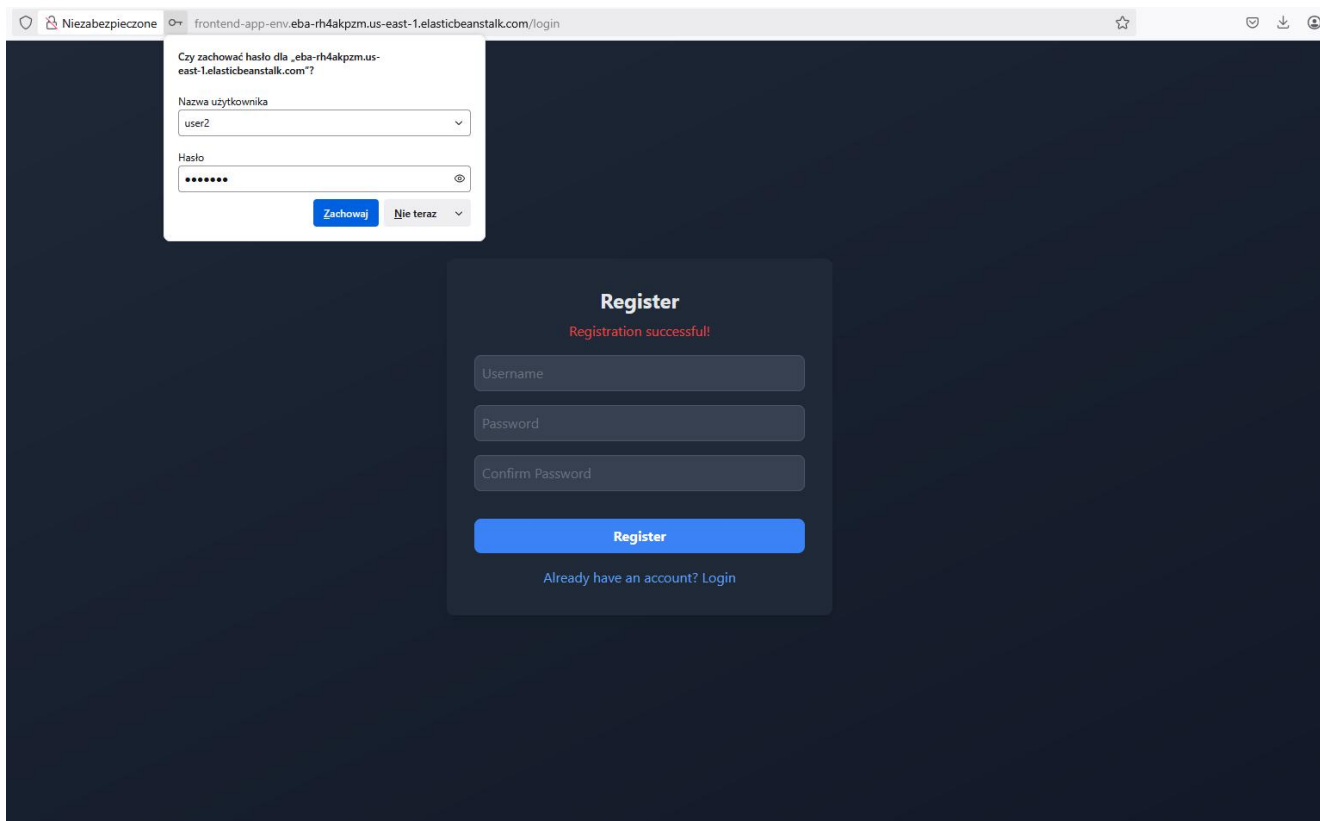
☐	App client name	Client ID
☐	user-pool-client	qi1s49ov9v00vingf7pf2pc7e

Rysunek 21 Klient puli użytkowników

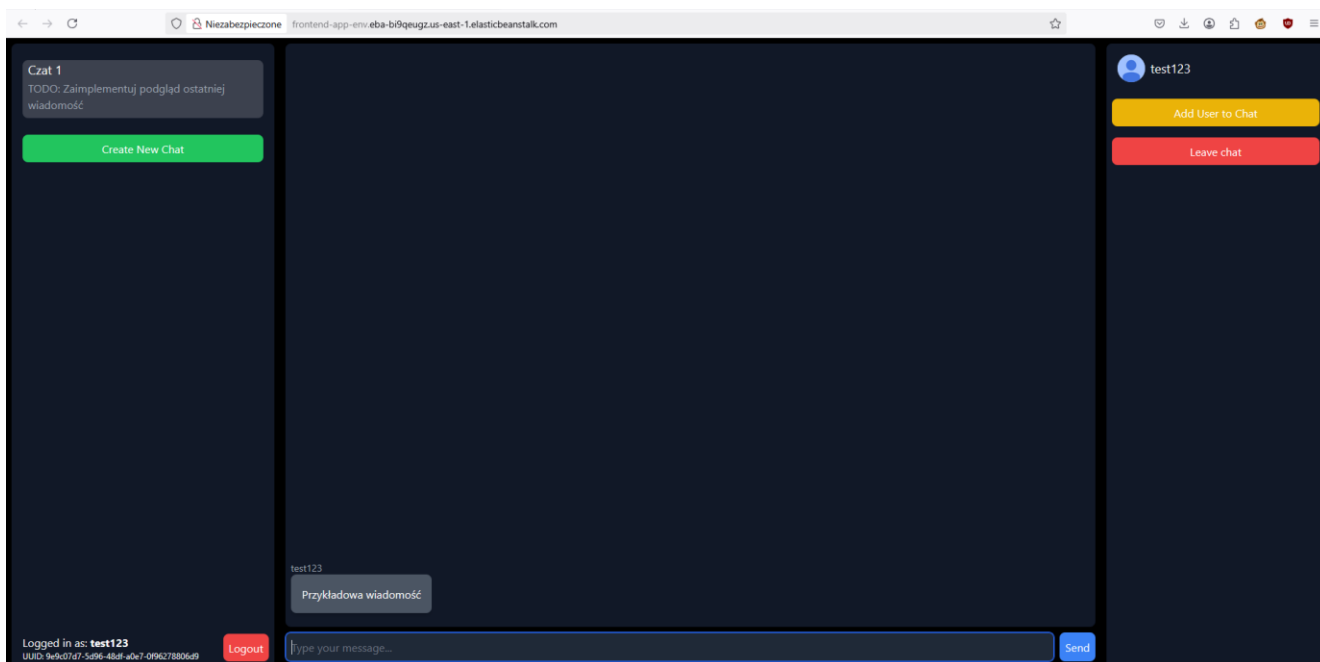
5. Prezentacja aplikacji



Rysunek 22 Ekran logowania



Rysunek 23 Rejestracja użytkownika



Rysunek 24 Widok czatu

6. Ręczne skonfigurowanie infrastruktury

W pierwszej kolejności utworzono S3 Bucket i wgrano pliki Dockerrun. Wszystkie ustawienia zostawiono domyślne. Pliki Dockerrun są jednakowe do tych z konfiguracji Terraform.

Create bucket [Info](#)

Buckets are containers for data stored in S3.

General configuration

AWS Region

US East (N. Virginia) us-east-1

Bucket type [Info](#)



General purpose

Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

Bucket name [Info](#)

264127.application

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or num

Copy settings from existing bucket - *optional*

Only the bucket settings in the following configuration are copied.

[Choose bucket](#)

Format: s3://bucket/prefix

Rysunek 25 Utworzenie S3 Bucket

Upload [Info](#)

Add the files and folders you want to upload to S3. To upload a file larger than 160GB, use the AWS CLI, AWS SDKs or Amazon S3 REST API. [Learn more](#)

Drag and drop files and folders you want to upload here, or choose [Add files](#) or [Add folder](#).

Files and folders (1 total, 223.0 B)

All files and folders in this table will be uploaded.

☐	Name	Folder
☐	Dockerrun.aws.json	-

☐	Type	Size
☐	application/json	223.0 B

[Remove](#)

[Add files](#)

[Add folder](#)

< 1 >

Destination [Info](#)

Destination

<s3://264127.application/frontend/>

► Destination details

Bucket settings that impact new objects stored in the specified destination.

► Permissions

Grant public access and access to other AWS accounts.

► Properties

Specify storage class, encryption settings, tags, and more.

[Cancel](#)

[Upload](#)

Rysunek 26 Wgranie Dockerrun.aws.json

Następnie utworzono Security Group dla serwera backend. Dodano dwie reguły dla ruchu przychodzącego, które zezwalają na połączenie http oraz https.

Create security group [info](#)

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name [info](#)

Name cannot be edited after creation.

Description [info](#)

VPC [info](#)

Inbound rules [info](#)

Type	Protocol	Port range	Source	Description - optional	
HTTP	TCP	80	Anywhere...		Delete
			0.0.0.0/0		
HTTPS	TCP	443	Anywhere...		Delete
			0.0.0.0/0		

Rysunek 27 Utworzenie backend-security-group

Oraz Security Group dla bazy danych zezwalający jedynie na połączenia urządzeń należących do wcześniej utworzonej backend-security-group.

Basic details

Security group name [info](#)

Name cannot be edited after creation.

Description [info](#)

VPC [info](#)

Inbound rules [info](#)

This security group has no inbound rules.

Outbound rules [info](#)

Type	Protocol	Port range	Destination	Description - optional	
PostgreSQL	TCP	5432	Custom		Delete
			sg-0917b52758bbf8063		
			sg-0917b52758bbf8063		

Rysunek 28 Utworzenie db-security-group

W kolejnym kroku utworzono bazę danych w usłudze AWS RDS. Wybrano bazę PostgreSQL, jako szablon wybrano Free Tier. W celach testowych baza posiada jedynie słabe dane uwierzytelniające dbuser:dbpassword. Przypisano jej wcześniej utworzone Security Group, a resztę ustawień pozostawiono domyślnie.

Create database [Info](#)

Choose a database creation method

☒ **Standard create**

You set all of the configuration options, including ones for availability, security, backups, and maintenance.

☐ **Easy create**

Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

Engine options

Engine type [Info](#)

☐ Aurora (MySQL Compatible)



☐ Aurora (PostgreSQL Compatible)



☐ MySQL



☒ PostgreSQL



Rysunek 29 Wybór silnika

Templates

Choose a sample template to meet your use case.

☐ **Production**

Use defaults for high availability and fast, consistent performance.

☐ **Dev/Test**

This instance is intended for development use outside of a production environment.

☒ **Free tier**

Use RDS Free Tier to develop new applications, test existing applications, or gain hands-on experience with Amazon RDS. [Info](#)

Availability and durability

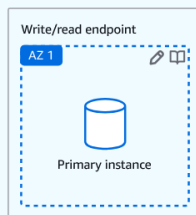
Deployment options [Info](#)

Choose the deployment option that provides the availability and durability needed for your use case. AWS is committed to a certain level of uptime depending on the deployment option you choose. Learn more in the [Amazon RDS service level agreement \(SLA\)](#).

☒ **Single-AZ DB instance deployment (1 instance)**

Creates a single DB instance without standby instances. This setup provides:

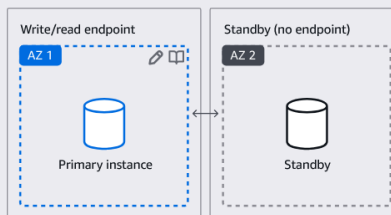
- 99.95% uptime
- No data redundancy



☐ **Multi-AZ DB instance deployment (2 instances)**

Creates a primary DB instance with a non-readable standby instance in a separate Availability Zone. This setup provides:

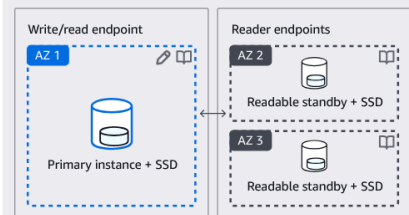
- 99.95% uptime
- Redundancy across Availability Zones



☐ **Multi-AZ DB cluster deployment (3 instances)**

Creates a primary DB instance with two readable standbys in separate Availability Zones. This setup provides:

- 99.95% uptime
- Redundancy across Availability Zones
- Increased read capacity
- Reduced write latency



Rysunek 30 Wybór szablonu

▼ Credentials Settings

Master username [Info](#)

Type a login ID for the master user of your DB instance.

dbuser

1 to 16 alphanumeric characters. The first character must be a letter.

Credentials management

You can use AWS Secrets Manager or manage your master user credentials.

☐ **Managed in AWS Secrets Manager - most secure**

RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☒ **Self managed**

Create your own password or have RDS create a password that you manage.

☐ **Auto generate password**

Amazon RDS can generate a password for you, or you can specify your own password.

Master password [Info](#)

••••••••

Password strength [Info](#)

Very weak

Minimum constraints: At least 8 printable ASCII characters. Can't contain any of the following symbols: / ' * @

Confirm master password [Info](#)

••••••••

Rysunek 31 Ustawienie nazwy użytkownika i hasła

VPC security group (firewall) [Info](#)

Choose one or more VPC security groups to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.

☒ **Choose existing**

Choose existing VPC security groups

☐ **Create new**

Create new VPC security group

Existing VPC security groups

Choose one or more options

backend-security-group X

Rysunek 32 Wybór security group

Databases (1)

DB identifier	Status	Role	Engine
text-messenger-db	Available	Instance	PostgreSQL

Rysunek 33 Uruchomiona baza danych

Następnie utworzono pulę użytkowników w AWS Cognito. Klient jest automatycznie tworzony przez AWS. Dodano także wcześniej wspomnianą funkcję lambda jako lambda trigger.

Define your application
Choose an application type and give it a name.

Application type | Info
Choose the type of application that you're developing. We will show example code for application like yours.

☒ **Traditional web application**
An application hosted on a webserver. Uses redirects and separate pages to display information. Examples are Java, Python, nodeJS.

☐ **Single-page application (SPA)**
A website with a single URL that updates content based on user interaction. Examples are JavaScript, Angular, React.

☐ **Mobile app**
An app built with a mobile SDK. Examples are Android, iOS.

☐ **Machine-to-machine application**
Platform-independent server-to-server communications without user interaction. Authorizes API access with OAuth 2.0 scopes.

Name your application | Info
Text-messenger
Names are limited to 128 characters or fewer. Names may only contain alphanumeric characters, spaces, and the following special characters: + = , . @ -

Configure options
You must make a few initial choices about the user pool that supports your application. To change these settings later, you must create a new user pool.

Options for sign-in identifiers | Info
Choose sign-in attributes. Usernames can be an email address, phone number, or a user-selected username. When you select only email and phone, users must select either email or phone as their username type. When username is an option, users can sign in with any options you select if they have provided a value for that option.

☐ Email
☐ Phone number
☒ Username

[Want to set up social, SAML, or OIDC sign-in?](#)

Required attributes for sign-up | Info
Choose any attributes that you want to require users to provide. With username alone, you must set email address or phone number as a required attribute.

Select attributes

email
User's preferred email address.

nickname
Casual name of the user that may or may not be the same as the given_name.

Options for sign-in identifiers and required attributes can't be changed after the app has been created.

Rysunek 34 Tworzenie puli użytkowników

Amazon Cognito > User pools > User pool - trjhwa > Extensions

Amazon Cognito
Current user pool: User pool - trjhwa
Overview
▼ Applications
App clients
▼ User management
Users
Groups
▼ Authentication
Authentication methods
Sign-in
Sign-up
Social and external providers
Extensions

Extensions
Lambda triggers (0)
Add and configure Lambda triggers for your user pool. Cognito can invoke Lambda functions in your account to customize authentication actions. Lambda triggers allow you to customize how you register and confirm users, authenticate users, send messages, and generate tokens.

Lambda triggers	Attached Lambda function	Trigger event version
No Lambda triggers		

Add Lambda trigger

Rysunek 35 Widok "Extensions" nowo utworzonej puli użytkowników

Lambda triggers

Trigger type [Info](#)

☒ **Sign-up**
Control sign-up, customize welcome messages, and migrate users.

☐ **Authentication**
Control sign-in, customize tokens, and log analytics events.

☐ **Custom authentication**
Create custom challenges and responses for user sign-in, such as a CAPTCHA or a security question.

☐ **Messaging**
Customize email and SMS messages, senders, and localization.

Sign-up
Choose a Lambda trigger type to customize sign-up. You can assign one function per trigger type.

☒ **Pre sign-up trigger**
Validate users when they sign up and customize their attributes.
 ☐ **Post confirmation trigger**
Customize welcome messages and log events for custom analytics.
 ☐ **Migrate user trigger**
Migrate a user from another directory when they sign in to your user pool.

Lambda function

Choose a Lambda function to assign to this trigger.

Assign Lambda function
Select a Lambda function in your account or create a new function.

Permission to invoke Lambda function
You are granting Amazon Cognito permission to invoke this Lambda function on your behalf. Amazon Cognito will add a resource-based policy statement to the function.

Cancel

Add Lambda trigger

Rysunek 36 Dodanie lambda trigger

W ostatnim kroku utworzono środowiska uruchamiającego aplikacje backend oraz frontend. Aplikacje są uruchamiane za pomocą obrazu Docker, używając wcześniej granych plików Dockerrun. Obie aplikacja utworzono jako pojedyncze instancje:

Review [Info](#)

Step 1: Configure environment

Environment information

Environment tier

Web server environment

Environment name

Backend-app-env

Platform

arn:aws:elasticbeanstalk:us-east-1::platform/Docker running on 64bit Amazon Linux 2023/4.4.4

Application name

backend-app

Application code

https://s3.us-east-1.amazonaws.com/264127.application/backend/Dockerrun.aws.json

Step 2: Configure service access

Service access [Info](#)

Configure the service role and EC2 instance profile that Elastic Beanstalk uses to manage your environment. Choose an EC2 key pair to securely log in to your EC2 instances.

Service role

arn:aws:iam::733338247552:role/LabRole

EC2 key pair

Lab-Keypair

EC2 instance profile

LabInstanceProfile

Rysunek 37 Konfiguracja backendu 1/4

Step 3: Set up networking, database, and tags

Networking, database, and tags [Info](#)

Configure VPC settings, and subnets for your environment's EC2 instances and load balancer. Set up an Amazon RDS database that's integrated with your environment.

Network

VPC

vpc-03f83890770cf601e

Public IP address

false

Instance subnets

subnet-0123874b5fd028997,subnet-03b4cc603aa595cb0

Tags

Key

Value

No tags

There are no tags defined

Rysunek 38 Konfiguracja backendu 2/4

Instance traffic and scaling [Info](#)

Customize the capacity and scaling for your environment's instances. Select security groups to control instance traffic. Configure the software that runs on your environment's instances by setting platform-specific options.

Instances		
IMDSv1 Deactivated	EC2 Security Groups sg-0917b52758bbf8063	
Capacity		
Environment type Single instance	Fleet composition On-Demand instance	On-demand base 0
On-demand above base 0	Capacity rebalancing Deactivated	Scaling cooldown 360
Processor type x86_64	Instance types t3.micro,t3.small	AMI ID ami-0a8ef704c8b49a12b

Rysunek 39 Konfiguracja backendu 3/4

Updates, monitoring, and logging [Info](#)

Define when and how Elastic Beanstalk deploys changes to your environment. Manage your application's monitoring and logging settings, instances, and other environment resources.

Monitoring						
System enhanced	Cloudwatch custom metrics - instance —	Cloudwatch custom metrics - environment —				
Log streaming Deactivated	Retention 7	Lifecycle false				
Updates						
Managed updates Activated	Deployment batch size 100	Deployment batch size type Percentage				
Command timeout 600	Deployment policy AllAtOnce	Health threshold Ok				
Ignore health check false	Instance replacement false					
Platform software						
Lifecycle false	Log streaming Deactivated	Proxy server nginx				
Logs retention 7	Rotate logs Deactivated	Update level minor				
X-Ray enabled Deactivated						
Environment properties						
<table><tr><th>Key</th><th>Value</th></tr><tr><td>DATABASE_URL</td><td>ecto://dbuser:dbpassword@text-messenger-db.ccsly5evhdma.us-east-1.rds.amazonaws.c...</td></tr></table>			Key	Value	DATABASE_URL	ecto://dbuser:dbpassword@text-messenger-db.ccsly5evhdma.us-east-1.rds.amazonaws.c...
Key	Value					
DATABASE_URL	ecto://dbuser:dbpassword@text-messenger-db.ccsly5evhdma.us-east-1.rds.amazonaws.c...					

Rysunek 40 Konfiguracja backendu 4/4

Review [Info](#)

Step 1: Configure environment

Edit

Environment information

Environment tier	Application name
Web server environment	frontend-app
Environment name	Application code
Frontend-app-env	https://s3.us-east-1.amazonaws.com/264127.application/frontend/Dockerrun.aws.json
Platform	
arn:aws:elasticbeanstalk:us-east-1::platform/Docker running on 64bit Amazon Linux 2023/4.4.4	

Step 2: Configure service access

Edit

Service access [Info](#)

Configure the service role and EC2 instance profile that Elastic Beanstalk uses to manage your environment. Choose an EC2 key pair to securely log in to your EC2 instances.

Service role	EC2 key pair	EC2 instance profile
arn:aws:iam::733338247552:role/LabRole	Lab-Keypair	LabInstanceProfile

Step 3: Set up networking, database, and tags

Edit

Networking, database, and tags [Info](#)

Configure VPC settings, and subnets for your environment's EC2 instances and load balancer. Set up an Amazon RDS database that's integrated with your environment.

Network		
VPC	Public IP address	Instance subnets
vpc-03f83890770cf601e	false	subnet-0123874b5fd028997,subnet-03b4cc603aa595cb0

Rysunek 41 Konfiguracja frontend 1/3

Step 4: Configure instance traffic and scaling

Edit

Instance traffic and scaling [Info](#)

Customize the capacity and scaling for your environment's instances. Select security groups to control instance traffic. Configure the software that runs on your environment's instances by setting platform-specific options.

Instances		
IMDSv1		
Deactivated		
Capacity		
Environment type	Fleet composition	On-demand base
Single instance	On-Demand instance	0
On-demand above base	Capacity rebalancing	Scaling cooldown
0	Deactivated	360
Processor type	Instance types	AMI ID
x86_64	t3.micro,t3.small	ami-0a8ef704c8b49a12b

Rysunek 42 Konfiguracja frontend 2/3

monitoring								
System enhanced	Cloudwatch custom metrics - instance —	Cloudwatch custom metrics - environment —						
Log streaming Deactivated	Retention 7	Lifecycle false						
Updates								
Managed updates Activated	Deployment batch size 100	Deployment batch size type Percentage						
Command timeout 600	Deployment policy AllAtOnce	Health threshold Ok						
Ignore health check false	Instance replacement false							
Platform software								
Lifecycle false	Log streaming Deactivated	Proxy server nginx						
Logs retention 7	Rotate logs Deactivated	Update level minor						
X-Ray enabled Deactivated								
Environment properties								
<table><tr><th>Key</th><th>Value</th></tr><tr><td>API_URL</td><td>http://backend-app-env.eba-w8kukq4m.us-east-1.elasticbeanstalk.com/api</td></tr><tr><td>SOCKET_URL</td><td>http://backend-app-env.eba-w8kukq4m.us-east-1.elasticbeanstalk.com/socket/websocket</td></tr></table>			Key	Value	API_URL	http://backend-app-env.eba-w8kukq4m.us-east-1.elasticbeanstalk.com/api	SOCKET_URL	http://backend-app-env.eba-w8kukq4m.us-east-1.elasticbeanstalk.com/socket/websocket
Key	Value							
API_URL	http://backend-app-env.eba-w8kukq4m.us-east-1.elasticbeanstalk.com/api							
SOCKET_URL	http://backend-app-env.eba-w8kukq4m.us-east-1.elasticbeanstalk.com/socket/websocket							

Rysunek 43 Konfiguracja frontend 3/3